

Project for 6th Semester of Bachelor of Information Technology

Sonar Rock Vs Mine Prediction



Submitted By:

Mausham Neupane (360379)

Mohammad Aarman (360381)

Tapendra Shahi (360400)

KIST College of Information Technology

Submitted To:

Faculty of Science and Technology

Purbanchal University

May 2026

STUDENT'S DECLARATION

We, the undersigned, hereby declare that the project titled “Sonar Rock Vs Mine Prediction”, submitted as a part of the academic requirement for the Sixth Semester of the Bachelor of Information Technology (B.I.T.) program under Purbanchal University, is our original work.

We affirm that this project has not been copied, plagiarized, or extracted from any previously published or submitted material. It has not been submitted elsewhere for any academic or professional purpose. All sources of information, if used, have been duly acknowledged.

We understand that any violation of academic integrity may lead to disqualification of the project.

S.N.	Name	Registration No.	Signature
1	Mausham Neupane	058-3-2-05288-2022	
2	Mohammad Aarman	058-3-2-05289-2022	
3	Tapendra Shahi	058-3-2-05310-2022	

Date: May 2026

RECOMMENDATION

This is to certify that the project work titled “Sonar Rock Vs Mine Prediction”, submitted by students of the Bachelor of Information Technology (B.I.T.) program at KIST College of Information Technology, has been completed as per the requirement of the Sixth Semester of the B.I.T. curriculum under Purbanchal University.

To the best of my knowledge, the submitted project is a genuine and original academic work carried out by the students with honesty and academic integrity. I, therefore, recommend this project for acceptance and official recognition by the University.

Deepak Khadka

Coordinator, B.I.T.

KIST College of Information Technology

May, 2026

CERTIFICATE

This is to certify that Mausham Neupane, Mohammad Aarman and Tapendra Shahi, students of Bachelor of Information Technology (B.I.T.) at KIST College of Information Technology, have successfully completed their academic project titled “Sonar Rock Vs Mine Prediction” as part of the course requirements for the Sixth Semester under the curriculum prescribed by Purbanchal University.

The project work was carried out during the academic session April 2026 – May 2026 and is a part of the continuous practical learning approach promoted by the B.I.T. program. The students demonstrated efforts in the planning, development and execution of the project, adhering to academic and technical standards.

Pankaj Jaiswal

Signature

Date signed

External Examiner

Bishal Khatri

Signature

Date signed

Supervisor

Deepak Khadka

Signature

Date signed

Coordinator, B.I.T.

ACKNOWLEDGEMENT

We are pleased to present the “Sonar Rock Vs Mine Prediction” as our Sixth semester project. We would like to express our sincere gratitude to all those who have supported and guided us throughout the duration of this project.

At the very first, a big thank you to our subject teacher Bishal Khatri, whose guidance, encouragement, and valuable feedback were crucial throughout the development of this report. We would also like to express our heartfelt gratitude to our B.I.T. coordinator Deepak Khadka for his valuable guidance and support in the preparation of the project.

We would like to take this opportunity to express our gratitude to all of our group members Mausham Neupane, Mohammad Aarman and Tapendra Shahi. The project would not have been successful without their cooperation and inputs.

We are also thankful to our colleagues and friends, for their constant support, constructive discussions and for always being there to offer help when needed. We are profoundly grateful to our family for their unconditional love, patience, and support.

Sincerely

Mausham Neupane

Mohammad Aarman

Tapendra Shahi

May, 2026

ABSTRACT

This project presents a machine learning-based application for classifying sonar signals as either Rock (R) or Mine (M) using a Logistic Regression model. The system is designed as a beginner-friendly implementation that demonstrates the complete machine learning workflow, including data preprocessing, model training, evaluation, model persistence, and real-time prediction.

The sonar dataset used in this project contains 60 numerical features representing sonar signal readings. These features are processed and passed through a trained Logistic Regression classifier to generate predictions with probability scores. The project also integrates a Flask-based backend API and a React frontend interface, allowing users to interact with the model through a simple and user-friendly web application.

The main objective of the project is to provide a practical understanding of binary classification and machine learning deployment. In addition to prediction through the web interface, the system also supports command-line interaction and API-based communication. The project highlights the importance of automation, data preprocessing, and model reusability in modern machine learning systems.

Overall, the system serves as both an educational project and a prototype for real-world machine learning applications involving signal classification and full-stack ML integration.

Keywords: Machine Learning, Logistic Regression, Sonar Signal Classification, Rock vs Mine Prediction, Binary Classification, Flask API, React Frontend, Scikit-learn, Data Preprocessing, Artificial Intelligence

TABLE OF CONTENTS

STUDENT’S DECLARATION	ii
RECOMMENDATION.....	iii
CERTIFICATE	iv
ACKNOWLEDGEMENT	v
ABSTRACT	vi
LIST OF FIGURES.....	ix
LIST OF ABBREVIATIONS	x
CHAPTER 1: BACKGROUND.....	1
1.1.Introduction.....	1
1.2.Objectives	1
1.3.Statement of Problem	1
1.4.Applications of Sonar Rock Vs Mine Prediction	1
1.5.Future Implementation of the project	2
1.6.Process Used in Project	2
1.7.Introduction to Software/Language used.....	3
CHAPTER 2: SIGNIFICANCE.....	5
2.1.Challenges in the Absence of the Proposed System.....	5
2.2.Proposed Solutions to Address the Identified Challenges.....	5
CHAPTER 3: LITERATURE REVIEW.....	6
3.1.Overview of the Problem Domain.....	6
3.2.Review of Machine Learning Concepts and Techniques	6
3.3.Review of Related Works and Existing Systems	6
3.4.Comparative Analysis of Reviewed Works	7
CHAPTER 4: METHODOLOGY	8
4.1. Requirement Analysis.....	8
4.1.1. Functional Requirement	8
4.1.2. Non- Functional Requirement	8
4.2. System Design	9
4.2.1. System Architecture	9
4.2.2. ML Pipeline Diagram	9
4.2.3. Data Flow Diagram (DFD).....	10
4.2.4. UseCase Diagram.....	10

4.2.5. Sequence Diagram	11
4.2.6. Algorithm.....	11
4.2.7. Flowchart of Model Training Process	15
4.3. Dataset Description and Preprocessing	15
4.4. Tools and Technologies.....	16
4.5. Development Model	16
4.6. Model Training and Evaluation Strategy.....	17
4.7. Testing Strategy	17
4.8. Deployment and Maintenance	18
4.9. Project Management	18
4.9.1. Gantt Chart	18
CHAPTER 5: SNAPSHOTS.....	19
CHAPTER 6: CONCLUSION	21
REFERENCES	22

LIST OF FIGURES

Figure 4.1: System Architecture	9
Figure 4.2: Pipeline Diagram	9
Figure 4.3: Data Flow Diagram (DFD).....	10
Figure 4.4: UseCase Diagram.....	10
Figure 4.5: Sequence Diagram	11
Figure 4.6: Flowchart	15
Figure 4.7: Gantt Chart	18
Figure 5.1: Homepage	19
Figure 5.2: Rock or Mine Prediction.....	20

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CLI	Command Line Interface
CSV	Comma Separated Values
HTTP	Hyper Text Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
ML	Machine Learning
UI	User Interface
UX	User Experience

CHAPTER 1: BACKGROUND

1.1. Introduction

Machine learning plays a crucial role in solving classification problems across various domains such as defense, healthcare, and finance. One such problem is identifying underwater objects using sonar signals.

This project focuses on building a binary classification model that predicts whether a sonar signal corresponds to a **rock** or a **mine**. The implementation is intentionally kept simple to help beginners understand the complete workflow of a machine learning system—from raw data to a deployed application.

1.2. Objectives

- a. To build a machine learning model for binary classification
- b. To understand the complete ML workflow
- c. To implement Logistic Regression for classification
- d. To deploy the model using a Flask API
- e. To create an interactive frontend using React
- f. To enable prediction through CLI, API, and UI

1.3. Statement of Problem

Identifying underwater objects using sonar signals is a challenging task due to the similarity in signal patterns reflected by different surfaces such as rocks and mines. Manual analysis of these signals can be time-consuming, error-prone, and requires domain expertise. Therefore, there is a need for an automated system that can accurately classify sonar readings into meaningful categories. This project aims to address this problem by developing a machine learning-based solution that takes 60 numerical sonar input values and predicts whether the signal corresponds to a rock or a mine. The goal is to create a simple, efficient, and reliable classification model that reduces human effort while demonstrating the practical implementation of binary classification using machine learning.

1.4. Applications of Sonar Rock Vs Mine Prediction

- a. Educational purposes:
This project helps beginners understand how a complete machine learning system works, from data processing to prediction.

- b. **Basic signal classification systems:**
It can be used as a simple example of how signals (like sonar data) can be automatically classified into categories.
- c. **Prototype ML deployment systems:**
It serves as a starting model for building real-world applications where machine learning models are deployed using APIs and user interfaces.

1.5. Future Implementation of the project

- a. **Educational purpose:**
Helps beginners understand the complete machine learning workflow, including data preprocessing, model training, evaluation, and deployment.
- b. **Signal classification:**
Demonstrates how sonar signal data can be automatically analyzed and classified into categories (rock or mine) using machine learning.
- c. **Real-world prototype:**
Acts as a basic prototype of real-time machine learning systems by combining a trained model with an API and user interface.
- d. **Full-stack ML integration:**
Shows how machine learning models can be integrated with backend (Flask API) and frontend (React) for interactive applications.

1.6. Process Used in Project

- a. **Data Collection:**
The sonar dataset (sonar_data.csv) is loaded, which contains 60 numerical features and a target label (Rock or Mine).
- b. **Data Preprocessing:**
The dataset is cleaned and split into input features (X) and output labels (Y).
Feature scaling is applied using Standard Scaler to normalize the data.
- c. **Train-Test Split:**
The dataset is divided into training and testing sets to evaluate model performance effectively.
- d. **Model Selection:**
Logistic Regression is selected as the machine learning algorithm for binary classification.

- e. **Model Training:**
The model is trained using the training dataset to learn patterns between sonar signals and their labels.
- f. **Model Evaluation:**
The trained model is tested using the test dataset and evaluated using accuracy and classification metrics.
- g. **Model Saving:**
The trained model pipeline is saved using Joblib for future reuse without retraining.
- h. **Prediction System:**
A prediction script is created to load the saved model and predict results from new input data.
- i. **API Development:**
A Flask API is built to expose the model as a web service for external access.
- j. **Frontend Integration:**
A React-based frontend is developed to allow users to input sonar data and view predictions visually.

1.7. Introduction to Software/Language used

1.7.1. Front-End

The front-end of this project is developed using React.js, Vite, and CSS to provide a simple, interactive, and user-friendly interface. It allows users to input sonar data easily and view prediction results in real time. The design focuses on clarity, responsiveness, and ease of use across different devices such as laptops, tablets, and desktops. The frontend communicates with the backend API to fetch prediction results and display them in a structured format.

1.7.2. Back-End

The back-end of the project is developed using Flask (Python), which handles the core machine learning prediction logic. It manages API requests, processes input data, loads the trained model, and returns prediction results. The backend ensures secure and efficient communication between the frontend and the machine learning model. It also handles data validation and error management during prediction requests.

1.7.3. Model and Data Processing

The machine learning component is built using Scikit-learn, where a Logistic Regression model is trained on the sonar dataset. The system processes raw input data by applying feature scaling using Standard Scaler before feeding it into the model. The trained model is saved using Joblib, allowing reuse without retraining. This ensures fast and consistent predictions for new input data.

1.7.4. Development Approach

The project follows a modular development approach, where each component (training, prediction, API, and frontend) is developed separately and then integrated. This approach improves maintainability, scalability, and debugging efficiency. The workflow follows a standard machine learning pipeline: data collection, preprocessing, model training, evaluation, deployment, and user interaction through a frontend interface.

CHAPTER 2: SIGNIFICANCE

2.1. Challenges in the Absence of the Proposed System

- a. Time-consuming manual analysis:
Without an automated system, analyzing sonar data manually requires significant time and effort.
- b. High probability of human error:
Handling and interpreting 60 numerical values manually can lead to incorrect classification.
- c. Lack of real-time decision making:
Manual or traditional methods cannot provide instant predictions, which limits practical usability.
- d. Inconsistent results:
Different individuals may interpret the same data differently, leading to unreliable outputs.
- e. Difficulty in handling large datasets:
Processing large volumes of sonar data manually is inefficient and not scalable.

2.2. Proposed Solutions to Address the Identified Challenges

- a. Automation using machine learning:
Implement a trained ML model to automatically classify sonar signals with higher accuracy.
- b. Use of Logistic Regression model:
Apply a reliable and simple classification algorithm to reduce complexity and improve performance.
- c. Real-time prediction system:
Integrate the model with an API to provide instant predictions for user inputs.
- d. Consistent and reliable output:
A trained model ensures the same result for the same input every time.
- e. Efficient data processing:
Use preprocessing techniques like scaling to improve performance and handle large data efficiently.
- f. User-friendly interface:
Provide CLI and web-based frontend for easy interaction with the system.

CHAPTER 3: LITERATURE REVIEW

3.1. Overview of the Problem Domain

The classification of sonar signals into different categories, such as rock and mine, is an important problem in the field of signal processing and pattern recognition. Sonar systems generate numerical data based on sound wave reflections from underwater objects. These signals often have similar patterns, making manual classification difficult and error-prone. Therefore, automated approaches using machine learning are widely used to improve accuracy and efficiency in such classification tasks.

3.2. Review of Machine Learning Concepts and Techniques

Machine Learning (ML) is a subset of Artificial Intelligence that enables systems to learn patterns from data and make predictions without being explicitly programmed. In classification problems, ML models are trained on labeled datasets to predict categories for new inputs.

For this project, Logistic Regression is used as the primary algorithm. It is a supervised learning technique suitable for binary classification tasks. Important concepts used include:

- i. Feature Scaling: Improves model performance by normalizing input data.
- ii. Train-Test Split: Divides data into training and testing sets for evaluation.
- iii. Model Evaluation Metrics: Includes accuracy, precision, recall, and F1-score.
- iv. Pipeline: Combines preprocessing and model training into a single workflow.

These techniques help in building an efficient and reliable classification model.

3.3. Review of Related Works and Existing Systems

Several studies have explored sonar signal classification using different machine learning algorithms such as Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Decision Trees, and Neural Networks. These approaches often achieve good accuracy but may involve higher computational complexity.

Traditional systems relied heavily on manual interpretation or rule-based methods, which were less accurate and time-consuming. Modern systems integrate machine learning models with web technologies to provide real-time predictions through APIs and user interfaces.

This project follows a similar approach by combining a Logistic Regression model with a Flask API and React frontend to create an accessible and interactive system.

3.4. Comparative Analysis of Reviewed Works

a. Logistic Regression:

Simple, fast, and easy to interpret; suitable for beginner-level projects and performs well on linearly separable data.

b. SVM:

Provides high accuracy but is more complex and computationally expensive.

c. KNN:

Easy to understand but slower for large datasets due to distance calculations.

d. Decision Trees:

Easy to visualize but may suffer from overfitting.

e. Neural Networks:

Highly powerful but require more data, tuning, and computational resources.

Compared to these methods, Logistic Regression offers a good balance between simplicity, performance, and efficiency, making it an ideal choice for this project.

CHAPTER 4: METHODOLOGY

4.1. Requirement Analysis

4.1.1. Functional Requirement

- a. Input of 60 sonar signal values
- b. Data validation and preprocessing
- c. Prediction of class as Rock (R) or Mine (M)
- d. Display of prediction result with confidence score
- e. Command Line Interface (CLI) support for prediction
- f. REST API for sending and receiving prediction requests
- g. Web-based user interface for input and result display
- h. Loading and reuse of trained machine learning mode

4.1.2. Non- Functional Requirement

- a. Performance
- b. Accuracy
- c. Usability
- d. Reliability
- e. Scalability

4.2. System Design

4.2.1. System Architecture

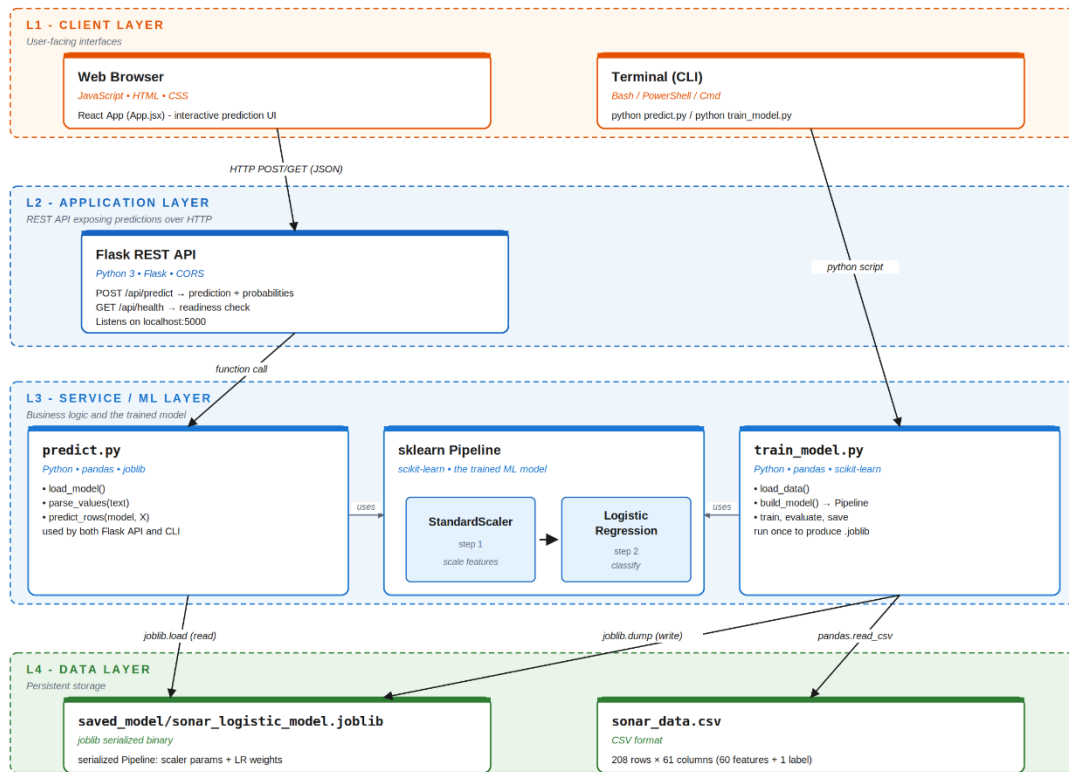


Figure 4.1: System Architecture

4.2.2. ML Pipeline Diagram

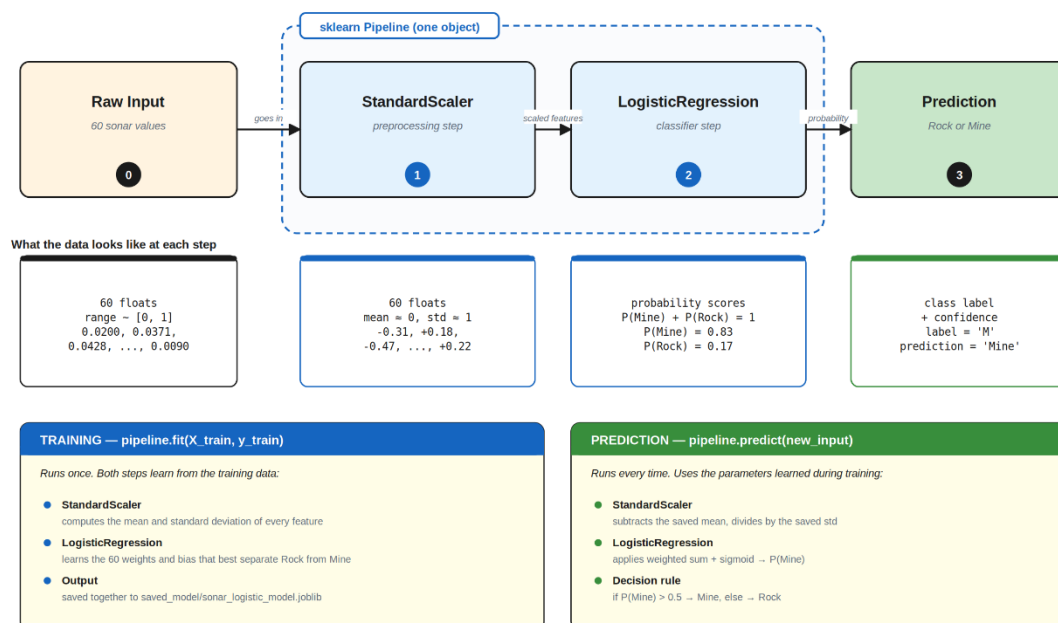


Figure 4.2: Pipeline Diagram

4.2.3. Data Flow Diagram (DFD)

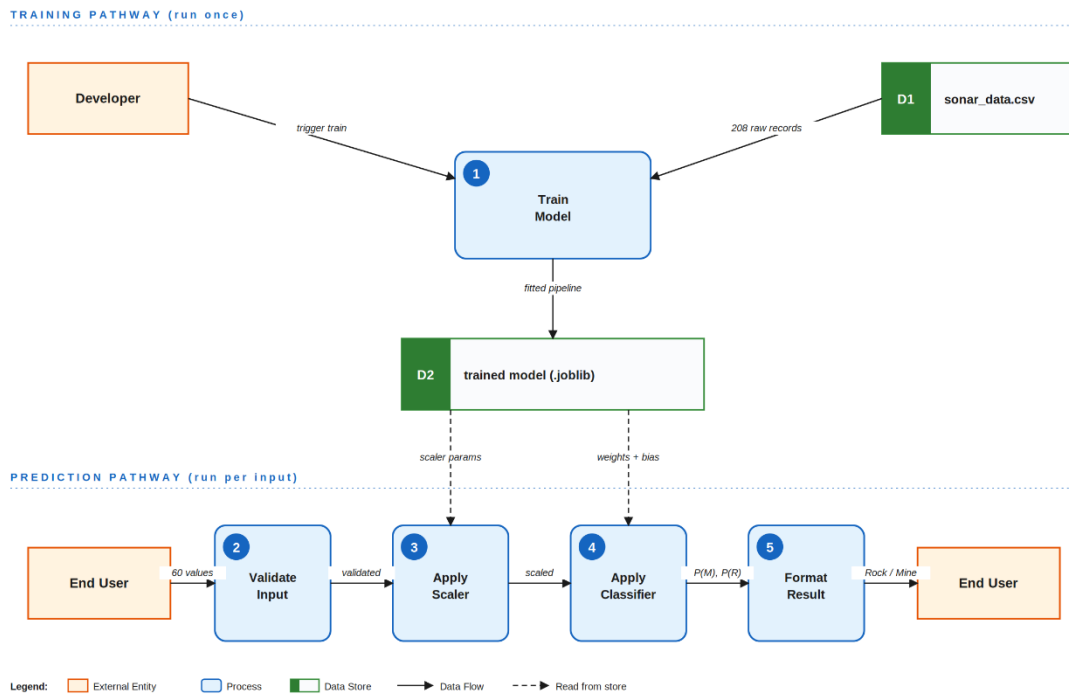


Figure 4.3: Data Flow Diagram

4.2.4. UseCase Diagram

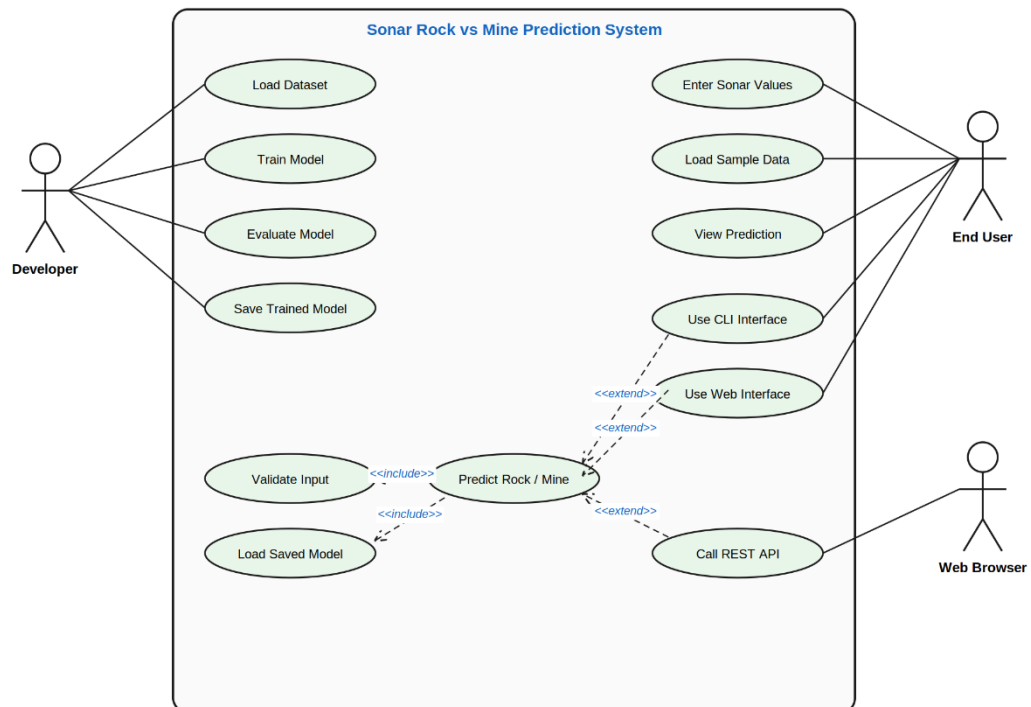


Figure 4.4: UseCase Diagram

4.2.5. Sequence Diagram

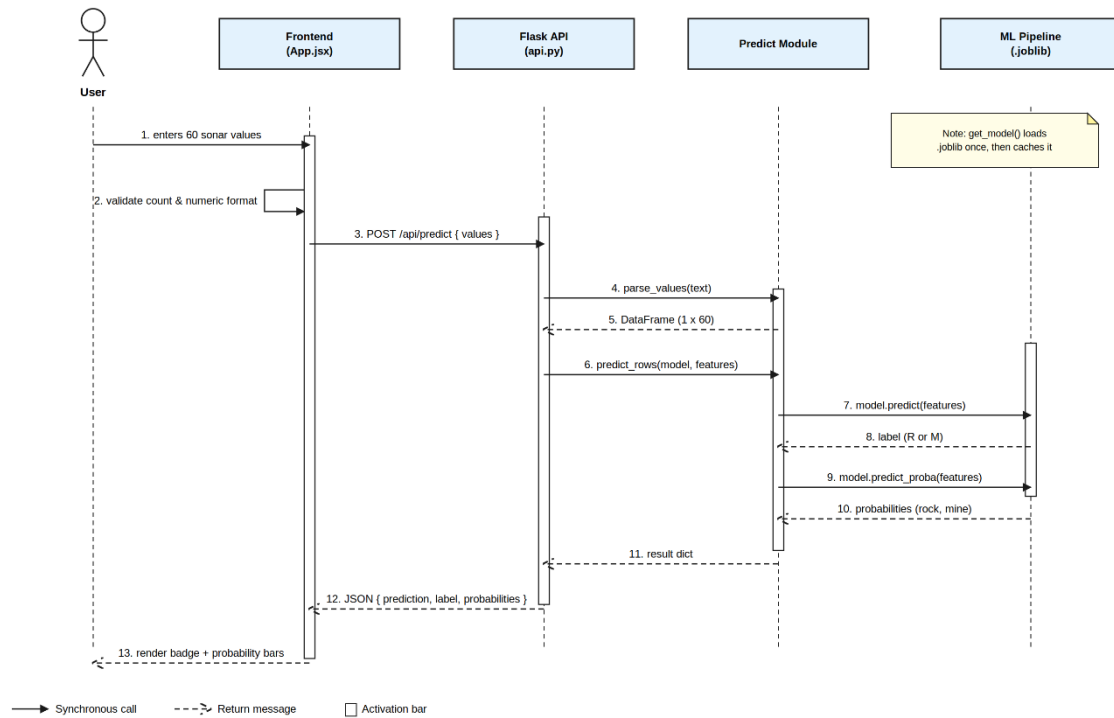


Figure 4.5: Sequence Diagram

4.2.6. Algorithm

Algorithm 1: Model Training

Purpose:

Train a Logistic Regression model on the sonar dataset and save it for later use.

Steps:

- Start
- Import required libraries (`'pandas'`, `'scikit-learn'`, `'joblib'`).
- Load the dataset `'sonar_data.csv'` into a Pandas DataFrame.
- Separate the dataset into:
 - Input features (X) — first 60 columns
 - Output label (Y) — last column (R or M)
- Split the data into training and testing sets:
 - 80% for training, 20% for testing
 - Use stratified split to keep class balance
 - Set `'random_state = 42'` for reproducibility
- Build a machine learning pipeline:

- i. Step A: `'StandardScaler'` to normalize the features
 - ii. Step B: `'LogisticRegression'` as the classifier
- g. Train (fit) the pipeline using the training data.
- h. Predict labels for the test set.
- i. Evaluate model performance:
 - i. Calculate accuracy
 - ii. Generate confusion matrix
 - iii. Generate classification report (precision, recall, F1-score)
- j. Print all evaluation results to the terminal.
- k. Save the trained pipeline to `'saved_model/sonar_logistic_model.joblib'` using `'joblib.dump'`.
- l. End

Algorithm 2: Command-Line Prediction

Purpose:

Use the saved model to predict whether a new sonar input is a Rock or a Mine through the terminal.

Steps:

- a. Start
- b. Import required libraries (`'argparse'`, `'joblib'`, `'pandas'`).
- c. Load the trained model from `'saved_model/sonar_logistic_model.joblib'`.
 - i. If the file does not exist, show an error and stop.
- d. Read the input source from command-line arguments:
 - i. If `'--values'` is given, read the comma-separated string.
 - ii. Else if `'--file'` is given, read 60 values from the CSV file.
 - iii. Else, ask the user to paste 60 values manually.
- e. Validate the input:
 - i. Check that exactly 60 numeric values are provided.
 - ii. If not, show an error message and stop.
- f. Convert the input values into a one-row Pandas DataFrame.
- g. Pass the input to the trained model:
 - i. Get the predicted label (R or M)
 - ii. Get the prediction probabilities for both classes
- h. Convert the label into a readable name:

- i. 'R' $\rightarrow$ Rock
 - ii. 'M' $\rightarrow$ Mine
- i. Display the result to the user:
 - i. Prediction (Rock or Mine)
 - ii. Mine probability
 - iii. Rock probability
- j. End

Algorithm 3: API Prediction

Purpose:

Expose the model as a web service so the frontend (or any HTTP client) can request predictions.

Steps:

- a. Start
- b. Initialize the Flask application.
- c. Define a global model variable (initially empty).
- d. When a request arrives at '/api/predict':
- e. Check if the model is already loaded in memory.
 - i. If not, load it from 'saved_model/sonar_logistic_model.joblib'.
- f. Read the request body as JSON.
- g. Extract the 'values' field from the JSON.
- h. Validate the input:
 - i. Must be a non-empty string.
 - ii. Must contain exactly 60 numeric values.
 - iii. If validation fails, return HTTP '400' with an error message.
- i. Parse the values into a Pandas DataFrame.
- j. Use the model to predict the class and probabilities.
- k. Build a JSON response containing:
 - i. 'prediction' (Rock or Mine)
 - ii. 'label' (R or M)
 - iii. 'probabilities' (mine and rock)
- l. Return the response with HTTP status '200'.
- m. When a request arrives at '/api/health':
 - i. Try to load the model.

- ii. If success, return `{ok: true}`.
- iii. If failure, return HTTP `503` with the error.
- n. Run the Flask server on `http://localhost:5000`.
- o. End

Algorithm 4: Frontend Prediction Flow

Purpose:

Allow the user to enter sonar values in a web interface and view the prediction.

Steps:

- a. Start
- b. Display the input form with a textarea for sonar values.
- c. The user types or pastes sonar values into the textarea.
- d. As the user types, split the input by commas, spaces, or line breaks.
- e. Count the number of values entered and show it as `X / 60`.
- f. When the user clicks the **Predict** button:
- g. Check that exactly 60 values are present.
 - i. If not, show an error message and stop.
- h. Check that every value is a valid number.
 - i. If not, show an error message and stop.
- i. Show a "Predicting..." loading state.
- j. Send a POST request to `/api/predict` with the values as JSON.
- k. Wait for the response from the Flask backend.
- l. If the response is successful:
 - i. Display the prediction (Rock or Mine) in a colored badge.
 - ii. Display the rock probability bar.
 - iii. Display the mine probability bar.
- m. If the response contains an error:
 - i. Show the error message to the user.
- n. The user can also:
 - i. Click **Load Sample** to fill the textarea with example values.
 - ii. Click **Clear** to reset the form and result.
- o. End

4.2.7. Flowchart of Model Training Process

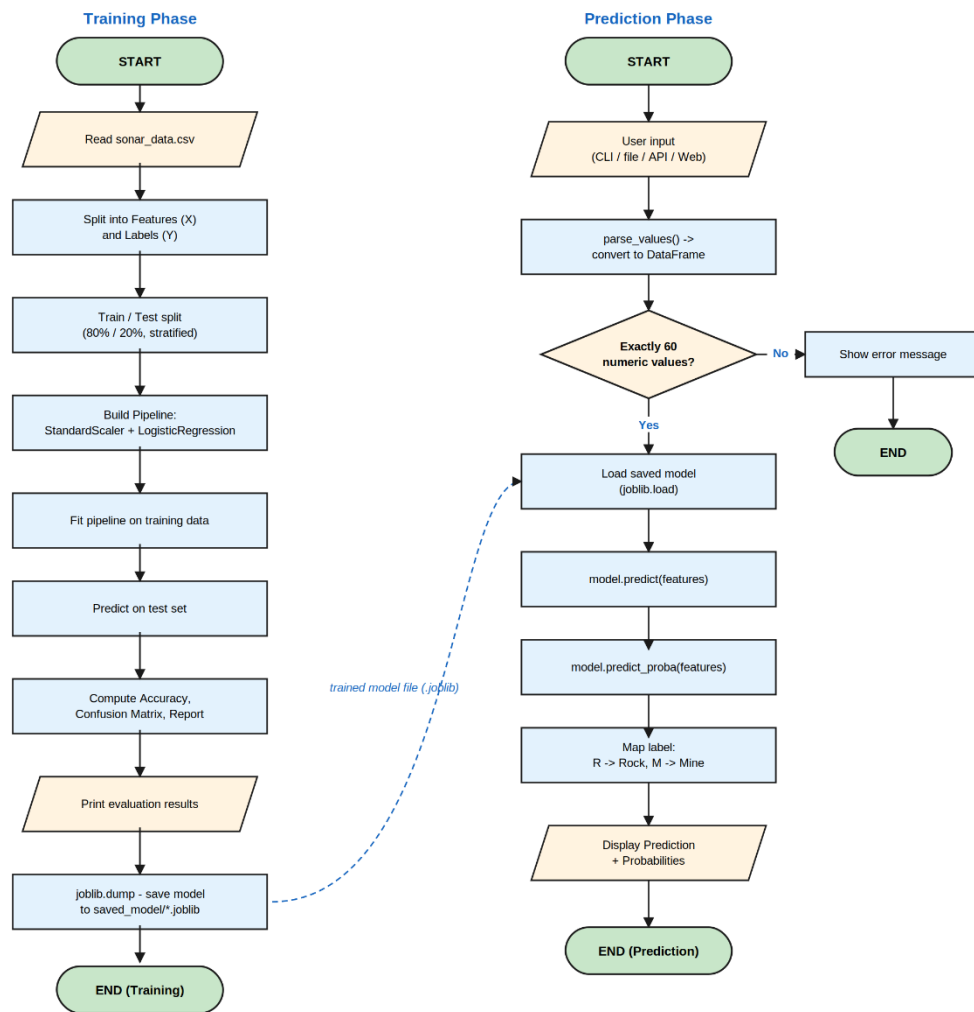


Figure 4.6: Flowchart

4.3. Dataset Description and Preprocessing

a. Dataset Overview:

The project uses the sonar dataset containing 208 samples with 60 numerical features and one target label (R for Rock, M for Mine).

b. Data Characteristics:

The dataset is clean, with no missing values, and feature values are within the range of 0 to 1.

c. Feature and Label Separation:

The first 60 columns are treated as input features, while the last column is used as the target label.

d. Data Splitting:

The dataset is divided into training and testing sets to evaluate model performance.

e. Feature Scaling:

StandardScaler is applied to normalize the data, improving the performance of the Logistic Regression model.

f. Data Validation:

Input data is validated to ensure that exactly 60 numerical values are provided for prediction.

4.4. Tools and Technologies

a. Programming Languages:

Python, JavaScript

b. Machine Learning Libraries:

Scikit-learn, Pandas, Joblib

c. Backend Technology:

Flask (Python)

d. Frontend Technology:

React.js (Vite), CSS

e. Development Tools:

Visual Studio Code (VS Code), Web Browser

f. Package Managers:

pip (Python), NPM (Node Package Manager)

4.5. Development Model

a. Modular Development Approach:

The project is developed in separate modules such as training, prediction, API, and frontend, which are later integrated.

b. Iterative Development:

Each component is developed and tested step-by-step to ensure correctness and improvement.

- c. **Machine Learning Pipeline:**
The development follows a structured ML workflow including data collection, preprocessing, model training, evaluation, and deployment.
- d. **Integration-Based Development:**
After building individual components, they are integrated to form a complete working system.

4.6. Model Training and Evaluation Strategy

- a. **Model Selection:**
Logistic Regression is chosen for binary classification due to its simplicity and effectiveness.
- b. **Training Process:**
The model is trained using the training dataset after applying feature scaling.
- c. **Pipeline Approach:**
A pipeline combining Standard Scaler and Logistic Regression ensures consistent preprocessing and training.
- d. **Evaluation Metrics:**
Model performance is evaluated using accuracy, confusion matrix, precision, recall, and F1-score.
- e. **Testing on Unseen Data:**
The model is tested on a separate test dataset to measure real-world performance.

4.7. Testing Strategy

- a. **Unit Testing:**
Individual components like data loading, prediction functions, and API endpoints are tested separately.
- b. **Integration Testing:**
Ensures proper interaction between frontend, backend, and machine learning model.
- c. **Input Validation Testing:**
Checks whether the system correctly handles invalid or incomplete input data.
- d. **API Testing:**
Verifies that API endpoints return correct responses for given inputs.
- e. **User Interface Testing:**
Ensures that the frontend works correctly and displays accurate results.

4.8. Deployment and Maintenance

a. Local Deployment:

The system is deployed locally using Flask for backend and Vite for frontend.

b. Model Deployment:

The trained model is saved and reused for predictions without retraining.

c. Future Deployment:

The system can be deployed on cloud platforms for wider accessibility.

d. Maintenance Strategy:

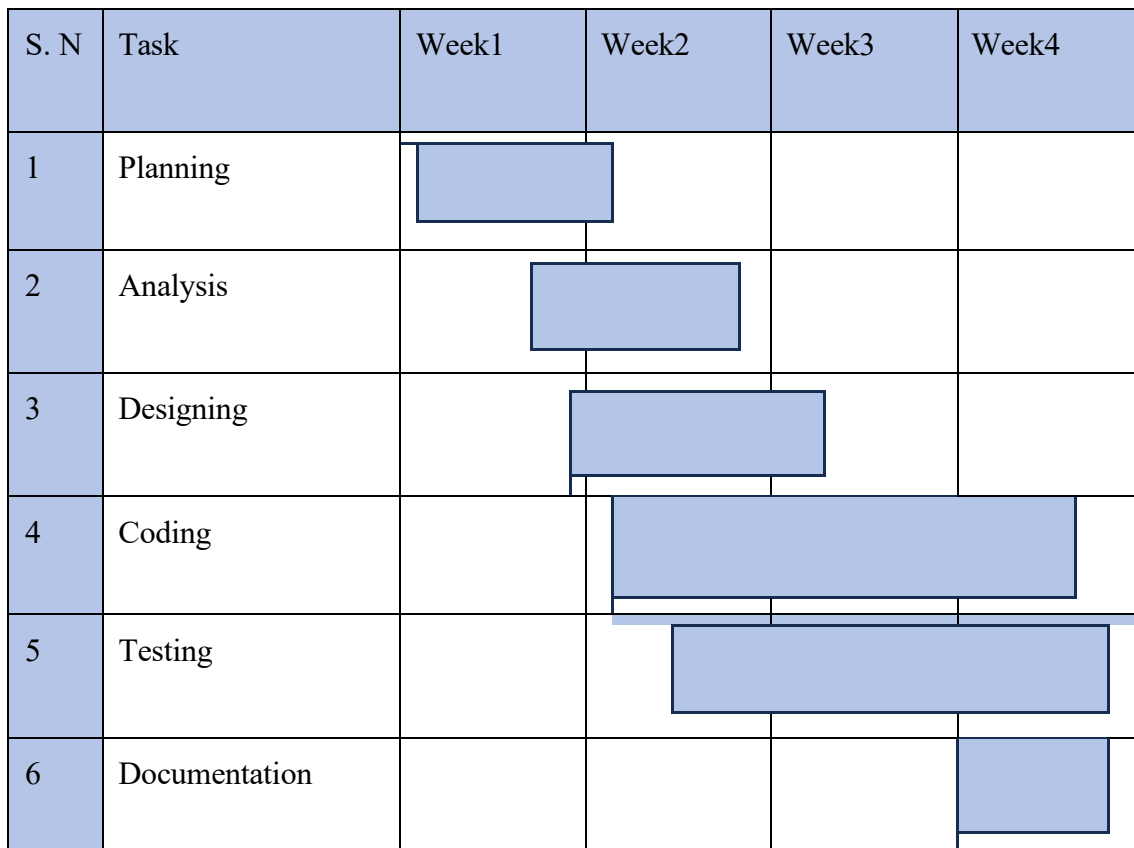
The model and system can be updated with new data and improved algorithms over time.

e. Bug Fixing and Updates:

Regular updates can be applied to improve performance and fix issues.

4.9. Project Management

4.9.1. Gantt Chart



Total time: 4 weeks (1 month)

Figure 4.7: Gantt Chart

CHAPTER 5: SNAPSHOTS

a. Homepage

SONAR SIGNAL CLASSIFICATION

Predict whether a new sonar reading is a rock or a mine.

Paste 60 sonar values, click predict, and the interface will return the model's classification with confidence scores.

NEW SONAR DATA

0/60 values

Input Values

Paste comma-separated values

0.0200, 0.0371, 0.0428, ...

You can separate values with commas, spaces, or line breaks. The model expects 60 numbers.

Predict

Load Sample

Clear

PREDICTION OUTPUT

Result

Ready for a prediction

Enter 60 sonar values on the left, then press **Predict** to see whether the model classifies the signal as rock or mine.

Figure 5.1: Homepage

b. Rock or Mine Prediction

SONAR SIGNAL CLASSIFICATION

Predict whether a new sonar reading is a rock or a mine.

Paste 60 sonar values, click predict, and the interface will return the model's classification with confidence scores.

NEW SONAR DATA

60/60 values

Input Values

Paste comma-separated values

0.0200,0.0371,0.0428,0.0207,0.0954,0.0986,0.1539,0.1601,0.3109,0.2111,0.1609,0.1582,0.2238,0.0645,0.0660,0.2273,0.3100,0.2999,0.5078,0.4797,0.5783,0.5071,0.4328,0.5550,0.6711,0.6415,0.7104,0.8080,0.6791,0.3857,0.1307,0.2604,0.5121,0.7547,0.8537,0.8507,0.6692,0.6097,0.4943,0.2744,0.0510,0.2834,0.2825,0.4256,0.2641,0.1386,0.1051,0.1343,0.0383,0.0324,0.0232,0.0027,0.0065,0.0159,0.0072,0.0167,0.0180,0.0084,0.0090,0.0032

You can separate values with commas, spaces, or line breaks. The model expects 60 numbers.

Predict

Load Sample

Clear

PREDICTION OUTPUT

Result

Rock

The model predicts this sonar signature is most likely a **rock**.

Rock

88.19%

Mine

11.81%

Figure 5.2: Rock or Mine Prediction

20

CHAPTER 6: CONCLUSION

This project successfully demonstrates a complete machine learning system for classifying sonar signals into Rock (R) or Mine (M) using a Logistic Regression model. The system effectively processes raw numerical data, applies feature scaling, and generates accurate predictions based on trained patterns.

The integration of multiple components such as machine learning model training, prediction script, REST API, and React-based frontend makes the system more practical and user-friendly. It also shows how a machine learning model can be deployed as a real-world application that supports different modes of interaction, including CLI, API, and web interface.

Overall, this project provides a strong foundation for understanding the end-to-end workflow of machine learning systems, from data preprocessing to deployment. It also highlights the importance of automation in reducing manual effort, improving accuracy, and enabling real-time decision-making.

In the future, the system can be further improved by using advanced algorithms, adding more features, and deploying it on cloud platforms for wider accessibility.

REFERENCES

1. J. Brownlee. (2016). Machine Learning Mastery with Python. Machine Learning Mastery. <https://machinelearningmastery.com>
2. Gorman, R. P., & Sejnowski, T. J. (1988). Analysis of hidden units in a layered network trained to classify sonar targets.
3. Neural Networks, 1(1), 75–89. This is the foundational reference for the dataset and shows you did proper research.
4. A. Géron. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media. <https://www.oreilly.com>
5. Scikit-learn Developers. (n.d.). *Scikit-learn Documentation*. <https://scikit-learn.org/stable/>
6. Meta AI. (n.d.). *React Documentation*. <https://react.dev/>
7. The Pandas Development Team. (n.d.). *Pandas Documentation*. <https://pandas.pydata.org/>